

Relatório técnico — análise do repositório `apache/superset`

Data da análise: 2026-05-14

Repositório: <https://github.com/apache/superset>

Clone local: `/home/ubuntu/superset`

Commit analisado: `d1e9a5d`

1) Contexto e escopo

Você redirecionou o estudo para o repositório público **Apache Superset**. A análise abaixo cobre:

- estrutura de diretórios e organização do monorepo;
- arquivos de configuração e empacotamento;
- stack tecnológica (backend, frontend, docs, websocket, CLI);
- arquitetura de código (APIs/views, comandos, DAOs, models);
- dependências e versões-chave;
- pontos de atenção e melhorias sugeridas.

2) Resultado da etapa “listar repositórios e localizar alvo”

No fluxo anterior, o repositório `safira_prisma` não estava acessível via integração.

Com o novo alvo fornecido por URL (`apache/superset`), o repositório foi clonado diretamente via Git público com sucesso.

3) Estrutura geral do repositório

Monorepo grande e modular. Principais diretórios:

- `superset/` → backend principal (Flask + Flask AppBuilder + SQLAlchemy)
- `superset-frontend/` → frontend React/TypeScript, plugins e packages internos
- `superset-websocket/` → sidecar WebSocket (Node/TypeScript + Redis stream)
- `superset-core/` → pacote Python core para extensões
- `superset-extensions-cli/` → CLI oficial para extensões
- `docs/` → portal de documentação (Docusaurus)
- `requirements/` → locks de dependência Python (`base.txt` , `development.txt`)
- `docker/` , `docker-compose*.yaml` , `Dockerfile` → ambientes de desenvolvimento/execução
- `helm/` → chart Kubernetes
- `tests/` → suíte Python

Métricas rápidas da árvore (aprox.)

- `superset-frontend` : **4548 arquivos** (predomínio TS/TSX)
- `superset` : **1421 arquivos** (predomínio Python)
- `tests` : **709 arquivos** (majoritariamente Python)
- `docs` : **476 arquivos**

Isso caracteriza um projeto enterprise de grande porte com múltiplos subprodutos.

4) Arquivos de configuração analisados

4.1 Backend Python

- `pyproject.toml`
- pacote: `apache_superset`
- `requires-python` = `">=3.10"`

- dependências principais incluem Flask, FAB, Celery, SQLAlchemy, Pandas, Numpy, Redis etc.
- **68 dependências base e 59 grupos opcionais** (muitos conectores de banco)
- `setup.py`
- versão derivada de `superset-frontend/package.json`
- injeta `version_info.json`
- define entrypoint CLI `superset`
- `requirements/base.txt` e `requirements/development.txt`
- lockfiles gerados por `uv pip compile`
- versões pinadas para reprodutibilidade

4.2 Frontend

- `superset-frontend/package.json`
- npm workspaces (`packages/*` , `plugins/*` , `src/setup/*`)
- React 18 + Ant Design + grande ecossistema de visualização (ECharts, deck.gl etc.)
- build com webpack, testes com Jest e Playwright
- `superset-frontend/.nvmrc` = `v22.22.0`

4.3 WebSocket Sidecar

- `superset-websocket/package.json`
- Node + TypeScript + Redis + ws + jsonwebtoken + hot-shots
- engines: Node `^22.22.0` , npm `^10.8.2`
- `superset-websocket/Dockerfile`
- build em duas etapas, runtime com `node:22-alpine`

4.4 Docs

- `docs/package.json` com Docusaurus 3.x, TS e scripts de geração de docs/API
- docs usam **yarn** (`docs/yarn.lock`)

4.5 Deploy/execução

- `Dockerfile` multi-stage (node + python, targets: `lean` , `dev` , `ci` , `showtime`)
- `docker-compose.yml` para desenvolvimento local com serviços:
- `superset` , `superset-worker` , `superset-worker-beat` , `db` (Postgres), `redis` , `nginx` , `superset-websocket` , `superset-node`
- `helm/superset` para Kubernetes

4.6 Prisma schema / package.json na raiz

- **Não há uso de Prisma** no repositório analisado.
- Não há um único `package.json` na raiz do monorepo; há `package.json` por subprojeto (`superset-frontend` , `superset-websocket` , `docs` , etc.).

5) Tecnologias e frameworks identificados

5.1 Backend

- **Python 3.10+**
- **Flask**
- **Flask AppBuilder** (estrutura administrativa/API/RBAC)
- **SQLAlchemy** + Alembic (migrations)
- **Celery** (tarefas assíncronas)

- Redis (cache/result backend/streams, dependendo da configuração)
- Marshmallow, Pydantic, Jinja2, PyArrow, Pandas etc.

5.2 Frontend

- **React 18 + TypeScript**
- Redux/React-Redux (partes legadas e modernas)
- Ant Design
- ECharts, deck.gl, mapbox/openlayers e diversos plugins de visualização
- Jest + Testing Library + Playwright + Storybook

5.3 Infra e DevEx

- Docker / Docker Compose / Helm
- GitHub Actions CI/CD
- Pre-commit + linters Python/TS
- Empacotamento Python via `setuptools` + lock de deps com `uv`

6) Estrutura de código e padrões arquiteturais

6.1 Backend (padrão em camadas)

A organização segue claramente uma separação por domínio e responsabilidade:

- **Views/APIs:** `superset/*/api.py` , `superset/views/*` (camada HTTP/REST)
- **Commands:** `superset/commands/*` (casos de uso/regras de negócio)
- **DAOs:** `superset/daos/*` (acesso a dados)
- **Models:** `superset/models/*` (ORM SQLAlchemy)

Exemplo observado:

- `superset/charts/api.py` (REST endpoints)
- `superset/commands/chart/create.py` (regra de criação)
- `superset/daos/chart.py` (operações de persistência)
- `superset/models/slice.py` (modelo de chart)

6.2 Inicialização da aplicação

- `superset/app.py` chama `SupersetAppInitializer` (`superset/initialization/__init__.py`)
- inicialização registra APIs, blueprints, middlewares, extensões e integração com Celery
- abordagem factory (`create_app`) com suporte a config customizada e app root custom

6.3 Frontend

- monorepo JS com packages/plugins internos
- bootstrap em `src/preamble.ts` (feature flags, setup de cliente, i18n, etc.)
- módulos por domínio (`dashboard` , `explore` , `SqlLab` , `database` , `embedded` ...)

6.4 WebSocket

- sidecar dedicado para eventos assíncronos
- conexão com Redis streams e multiplexação por canais/sessão
- foco em envio de eventos de execução assíncrona para UI

7) Dependências e versões (recorte principal)

7.1 Python (pyproject)

- Flask `>=2.2.5, <4.0.0`
- Flask-AppBuilder `>=5.2.1, <6.0.0`
- Celery `>=5.3.6, <6.0.0`
- SQLAlchemy `>=1.4, <2`
- Pandas `[excel] >=2.1.4, <2.4`
- NumPy `>1.23.5, <2.3`
- Redis `>=5.0.0, <6.0`
- PyArrow `>=16.1.0, <21`

7.2 Frontend

- React `^18.2.0`
- Antd `^5.26.0`
- ECharts `^5.6.0`
- deck.gl `~9.2.5`
- Webpack `^5.106.2`
- Jest/Playwright/Storybook presentes

7.3 WebSocket

- ws `^8.20.1`
- ioredis `^5.10.1`
- jsonwebtoken `^9.0.3`
- winston `^3.19.0`

7.4 Lockfiles

- Python: `requirements/base.txt` , `requirements/development.txt`
- Frontend/WebSocket/SDK: `package-lock.json`
- Docs: `yarn.lock`

8) Pontos fortes observados

- arquitetura modular e separação de responsabilidades consistente;
- suporte extenso a bancos de dados e conectores opcionais;
- infraestrutura madura de CI/CD e automação de qualidade;
- ecossistema de extensões (core + CLI + SDK embutido);
- estratégia multi-ambiente (dev/lean/ci/showtime) via Docker;
- grande cobertura de testes e documentação detalhada.

9) Possíveis problemas / oportunidades de melhoria

9.1 Complexidade operacional elevada

Projeto muito grande com várias “frentes” (backend, frontend, websocket, docs, extensões), o que aumenta curva de aprendizado e tempo de onboarding.

Sugestão: manter um mapa arquitetural “quick-start para contribuidores” com fluxos por persona (backend, frontend, docs, connector).

9.2 Mistura de ferramentas de pacote JS

- frontend/websocket usam npm;
- docs usam yarn.

Isso pode aumentar atrito em times e CI.

Sugestão: avaliar convergência gradual (ou formalizar claramente a justificativa por subprojeto).

9.3 Arquivos muito extensos (potencial dívida técnica)

- `superset/config.py` (muito grande)
- `superset/charts/api.py` (muito grande)
- `superset/models/core.py` (muito grande)

Sugestão: refatoração progressiva em módulos menores por domínio/tema.

9.4 Dependências e compatibilidade

Há muitos limites de versão e comentários sobre incompatibilidades conhecidas (ex.: marshmallow 4), o que exige manutenção contínua.

Sugestão: roadmap periódico de upgrade de dependências críticas com testes de regressão focados.

9.5 Segurança por configuração

`superset/config.py` mantém defaults que **precisam** ser sobrescritos em produção (ex.: `SECRET_KEY`). O próprio `compose` alerta sobre isso.

Sugestão: validações de startup mais “fail-fast” em produção quando secrets inseguros forem detectados.

9.6 Observação de supply chain

No frontend há dependência direta por URL tarball (`xlsx` via CDN), que pode exigir governança adicional.

Sugestão: avaliar política interna de dependências remotas e checagem de integridade.

10) Conclusão

O `apache/superset` é um monorepo robusto e enterprise-grade, com:

- backend Python altamente extensível,
- frontend React/TS avançado com ecossistema de plugins,
- sidecar websocket para eventos assíncronos,
- pipeline de build/test/deploy maduro.

A principal atenção para times novos é **gestão de complexidade** (arquitetura ampla + múltiplos subprojetos + forte matriz de dependências). A base técnica, no entanto, é sólida e bem organizada para evolução contínua.